TRABAJO PRÁCTICO INTEGRADOR FINAL

LinuxGuard NOC

Sistema de monitoreo de servidores GNU/Linux

MATERIA
Práctica Profesionalizante III

CARRERA
Tecnicatura Superior en Soporte de Infraestructura de TI

DOCENTE
Carlos Contreras

INTEGRANTES
Franco Carrasco
Lucio Rivera

AÑO 2026

Tecnologías utilizadas

Node.js – JavaScript – HTML5 – CSS3 – Bootstrap 5 – CORS – JSON – API REST – Express – Socket.IO – systeminformation – pnpm – Git – GitHub – GitHub Helpers – systemd

LinuxGuard NOC es una aplicación cliente-servidor para monitorear un equipo GNU/Linux. El backend obtiene métricas del sistema, expone una API REST y transmite actualizaciones en tiempo real mediante Socket.IO. El frontend se sirve desde el mismo backend mediante la carpeta public.

BACK END EN NODE.JS

Debemos disponer de un servidor Debian previamente instalado y con acceso a la red.

1- Verificar Node.js, npm y pnpm

En la máquina virtual utilizada para la demostración final se verificaron las herramientas instaladas:

```
node -v
npm -v
pnpm -v
```

Versiones verificadas en la VM de prueba:

```
Node.js: v22.23.0
npm: 10.9.8
pnpm: 11.5.1
```

2- Clonar el proyecto desde GitHub

El repositorio principal del proyecto es:

```
https://github.com/frxxn-linuxadmin/linuxguard-noc.git
```

Clonar el repositorio y seleccionar la rama principal:

```
git clone https://github.com/frxxn-linuxadmin/linuxguard-noc.git
cd linuxguard-noc
git switch main
```

3- Ubicar la aplicación en /opt

Para la instalación final se utiliza /opt/linuxguard-noc como ubicación de la aplicación.

```
sudo mv /home/fran/linuxguard-noc /opt/linuxguard-noc
sudo chown -R fran:fran /opt/linuxguard-noc
```

Ruta final del proyecto:

```
/opt/linuxguard-noc
```

4- Instalar las dependencias

Ingresar al backend:

```
cd /opt/linuxguard-noc/backend
```

Instalar las dependencias declaradas en package.json:

```
pnpm install
```

Dependencias principales utilizadas:

- express: framework web utilizado para la API REST y para servir el frontend.
- socket.io: comunicación en tiempo real entre servidor y navegador.
- systeminformation: obtención de métricas del sistema GNU/Linux.
- cors: control de solicitudes entre distintos orígenes.

5- Estructura principal del proyecto

```
linuxguard-noc/
├── backend/
│   ├── config/
│   │   └── thresholds.json
│   ├── data/
│   │   └── events.json
│   ├── public/
│   │   ├── css/
│   │   │   └── style.css
│   │   ├── js/
│   │   │   └── app.js
│   │   └── index.html
│   ├── services/
│   │   ├── alertService.js
│   │   ├── configService.js
│   │   ├── eventService.js
│   │   ├── serviceMonitor.js
│   │   └── systemService.js
│   ├── package.json
│   ├── pnpm-lock.yaml
│   └── server.js
└── README.md
```

6- Ejecutar el servidor

Para una prueba manual:

```
cd /opt/linuxguard-noc/backend
pnpm start
```

El servidor escucha en el puerto 3000 y acepta conexiones desde las interfaces de red mediante 0.0.0.0.

7- Endpoints principales

Método	Ruta	Función
GET	/api/health	Comprueba que el backend está funcionando.
GET	/api/sistema	Devuelve métricas, procesos, servicios y alertas.
GET	/api/eventos	Devuelve el historial persistente de eventos.

GET	/api/config/umbrales	Consulta los umbrales configurados.
PUT	/api/config/umbrales	Guarda nuevos umbrales validados.
Socket.IO	metricas	Envía métricas en tiempo real al frontend.

Ejemplo de consulta desde un navegador:

```
http://IP_DEL_SERVIDOR:3000/api/sistema
```

8- Métricas obtenidas

La biblioteca systeminformation se utiliza para obtener, entre otros datos:

- CPU: fabricante, modelo, núcleos, velocidad y porcentaje de uso.
-
- Memoria RAM total, usada y disponible.
- Uso de discos.
- Interfaces y tráfico de red.
- Hostname, sistema operativo, kernel y arquitectura.
- Tiempo de actividad.
- Procesos con mayor consumo.

9- Alertas, eventos y configuración

LinuxGuard NOC compara los valores de CPU, memoria y disco con umbrales configurables. Los estados posibles son normal, advertencia y crítico.

Archivo de configuración:

```
/opt/linuxguard-noc/backend/config/thresholds.json
```

Historial de eventos:

```
/opt/linuxguard-noc/backend/data/events.json
```

FRONT END

El frontend no se ejecuta como un proyecto separado. Está integrado dentro del backend en la carpeta public y Express lo publica directamente.

Ubicación:

```
/opt/linuxguard-noc/backend/public
```

Acceso principal desde cualquier equipo de la misma red:

```
http://IP_DEL_SERVIDOR:3000/
```

Tecnologías utilizadas en el frontend:

- HTML5 para la estructura.

- CSS3 para el diseño personalizado.
- Bootstrap 5 para formularios, botones, tablas y utilidades responsive.
- JavaScript para la lógica del dashboard.
- Canvas para los gráficos.
- Socket.IO Client para actualización en tiempo real.

El dashboard muestra CPU, RAM, disco, red, uptime, procesos, servicios, alertas, historial de eventos y configuración de umbrales.

SERVICIO SYSTEMD

LinuxGuard NOC se configuró como servicio de Debian para iniciar automáticamente al arrancar el sistema.

Archivo:

```
/etc/systemd/system/linuxguard.service
```

Configuración utilizada:

```
[Unit]
Description=LinuxGuard NOC - Sistema de monitoreo GNU/Linux
After=network.target

[Service]
User=fran
WorkingDirectory=/opt/linuxguard-noc/backend
ExecStart=/usr/bin/node /opt/linuxguard-noc/backend/server.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Comandos para habilitar e iniciar el servicio:

```
sudo systemctl daemon-reload
sudo systemctl enable linuxguard.service
sudo systemctl start linuxguard.service
sudo systemctl status linuxguard.service
```

Se realizó una prueba reiniciando la VM y se verificó que LinuxGuard NOC vuelve a iniciar automáticamente sin ejecutar pnpm start manualmente.

DATOS DEL SERVIDOR DE PRUEBA

Sistema operativo	Debian GNU/Linux 12 (bookworm)
Hostname	linuxguard-noc
Arquitectura	x86_64
Ruta de instalación	/opt/linuxguard-noc
Puerto de la aplicación	3000
Gestor del servicio	systemd

GITHUB Y GITHUB HELPER

Repositorio:

```
https://github.com/frxxn-linuxadmin/linuxguard-noc
```

Revisión del proyecto mediante GitIngest:

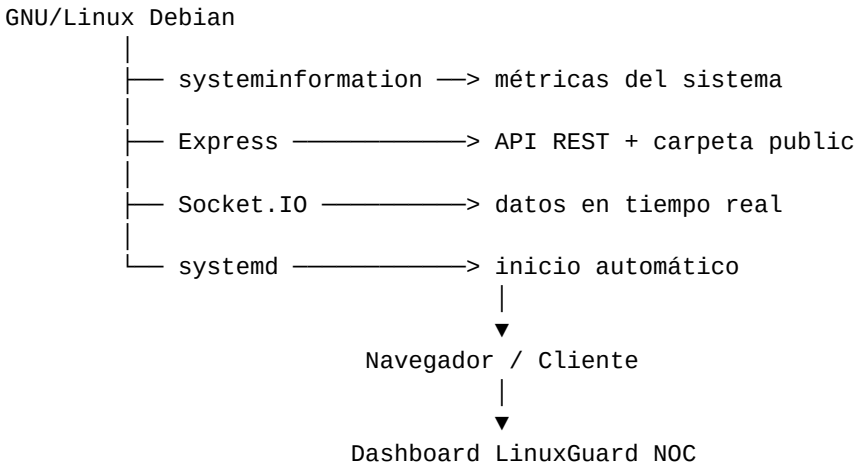
```
https://gitingest.com/frxxn-linuxadmin/linuxguard-noc
```

ORGANIZACIÓN DEL TRABAJO

El desarrollo se realizó utilizando ramas individuales y una rama principal de integración.

- franco: backend, API, Socket.IO, alertas, eventos, configuración, integración y despliegue.
- lucio: frontend, dashboard, CSS, JavaScript, gráficos e integración visual.
- main: versión integrada, probada y utilizada para la entrega.

RESUMEN DE FUNCIONAMIENTO



LinuxGuard NOC monitorea el mismo equipo GNU/Linux donde se ejecuta el backend. No realiza descubrimiento automático de múltiples servidores de la red.

APLICACIÓN MÓVIL / APK

El frontend de LinuxGuard NOC fue empaquetado como una aplicación Android mediante HTML2Apk Converter. La aplicación móvil utiliza un WebView para cargar el frontend HTML, CSS y JavaScript y se conecta por red al backend Node.js ejecutado en el servidor Debian.

Preparación del frontend para el APK:

- Se realizó una copia independiente del contenido de backend/public/.
- Bootstrap 5 se descargó localmente para evitar dependencia del CDN dentro del APK.
- Se mantuvieron las rutas relativas de index.html, css/, js/ y vendor/.
- Se configuró permiso INTERNET en el manifiesto generado.
- Se permitió tráfico HTTP en la red local para conectar con http://IP_DEL_SERVIDOR:3000.
- Se habilitó JavaScript y acceso desde archivos locales al backend mediante WebView.

Datos utilizados para generar el APK:

Nombre de la app	LinuxGuard NOC
Package	com.example.linuxguardnoc

Tipo de salida	APK firmado
Versión	1.0 y 1.1 (final)
Permiso principal	INTERNET
Orientación	Unspecified
Target Android	Android 16 / API 36
Firma	Keystore existente de prueba

Archivos APK conservados:

LinuxGuard_NOC-v1.0-signed.apk

LinuxGuard_NOC-v1.1-final.apk

El APK fue instalado y probado correctamente en un dispositivo Samsung con Android 16. Desde la aplicación se configuró la dirección del backend mediante una URL de red local y se verificó la recepción de métricas reales en tiempo real.

Validaciones realizadas desde Android:

- Conexión al backend Debian.
- Actualización de CPU, RAM y disco.
- Visualización de procesos y servicios.
- Recepción de métricas mediante API REST y Socket.IO.
- Visualización correcta del dashboard responsive.
- Uso del mismo sistema de alertas, historial y configuración de umbrales.

VERSIONADO DE LA APLICACIÓN

Se conservaron dos versiones del APK como parte de la evolución técnica del proyecto.

v1.0: primera versión funcional. Incluía la visualización de temperatura de CPU mediante systeminformation.

v1.1 (versión final): se retiró la temperatura de CPU del backend, las alertas, la configuración y la interfaz.

Motivo técnico: en el entorno virtualizado utilizado para la demostración no se dispone de un sensor térmico accesible y fiable. La lectura podía devolver 0 °C o un valor no disponible, lo que podía interpretarse erróneamente como una medición real.

Decisión adoptada: se eliminó esa métrica para evitar presentar información engañosa. Se conservaron ambos APK para documentar la evolución del sistema y la corrección realizada a partir de las pruebas.

ESTADO FINAL DEL PROYECTO

LinuxGuard NOC quedó funcionando como una solución cliente-servidor completa. El servidor Debian ejecuta el backend Node.js como servicio systemd y expone la API REST, Socket.IO y el frontend web. El sistema puede consumirse desde un navegador en la red local o desde la aplicación Android empaquetada como APK.

- Backend desplegado en Debian y alojado en /opt/linuxguard-noc.
- Inicio automático mediante linuxguard.service.
- Frontend integrado en backend/public/.
- Bootstrap 5 utilizado en la interfaz.
- API REST y Socket.IO funcionando.
- GitHub y GitIngest documentados.
- README actualizado.
- APK Android v1.0 y v1.1 generados y firmados; v1.1 corresponde a la versión final recomendada.
- Prueba real de conexión Android → Debian completada correctamente.